



Control de Versiones con Git: comandos esenciales y creación de ramas

Programa: Análisis y Desarrollo de Software

Programa de Formación	Tecnólogo en Análisis y Desarrollo de Software
Nombre del Proyecto	Desarrollo de Software para la Gestión de Procesos Empresariales
Fase del Proyecto	Análisis y Ejecución
Actividad de Proyecto	Control de versiones y trabajo colaborativo con Git
Competencia	Aplicar herramientas de control de versiones en el desarrollo de software conforme a metodologías y buenas prácticas de la industria
Resultados de Aprendizaje	Utilizar comandos básicos de Git para el seguimiento, registro y gestión de versiones de un proyecto de software, incluyendo la creación y manejo de ramas
Duración de la Guía	6 horas.

Guía de Aprendizaje — Control de Versiones con Git

1. Introducción

En el desarrollo de software actual, ningún proyecto profesional se construye sin un sistema de control de versiones. Git se ha convertido en la herramienta estándar de la industria para registrar la historia de un proyecto, coordinar el trabajo de varias personas sobre el mismo código y recuperar versiones anteriores cuando algo sale mal.

Esta guía está diseñada para que usted, como aprendiz del programa de Análisis y Desarrollo de Software, comprenda y practique los comandos fundamentales de Git — git clone, git status, git add ., y git log — y aprenda a crear y utilizar ramas (branches) como base del trabajo colaborativo. La guía se desarrolla en cuatro momentos: reflexión inicial, contextualización teórica, apropiación práctica del conocimiento y una actividad de transferencia en la que usted sustentará ante el instructor lo aprendido.

Requisitos previos

Tener instalado Git en su equipo (verificable con el comando `git --version`), contar con una cuenta activa en GitHub y tener un editor de código instalado, preferiblemente Visual Studio Code.

2. Actividades de Reflexión

Antes de entrar en la práctica técnica, es fundamental que usted reflexione sobre la necesidad y el valor del control de versiones en el desarrollo de software. Responda de forma individual y por escrito las siguientes preguntas en su bitácora de aprendizaje. No existen respuestas incorrectas: el objetivo es activar su pensamiento crítico frente al problema que Git resuelve.

2.1 Preguntas orientadoras

1. Imagine que usted y dos compañeros están construyendo la misma aplicación y se envían el código por correo electrónico o por USB cada vez que hacen un cambio. Describa al menos tres problemas que podrían presentarse con esta forma de trabajo.
2. ¿Qué cree que pasaría si, después de tres semanas de trabajo, se diera cuenta de que un cambio realizado hace cinco días dañó una funcionalidad que antes funcionaba correctamente? ¿Cómo resolvería esta situación sin una herramienta de control de versiones?

3. ¿Por qué cree que las empresas de tecnología exigen conocimientos de Git como requisito básico para casi cualquier vacante de desarrollo de software, sin importar el lenguaje de programación que se utilice?
4. A partir de su experiencia personal (académica o laboral), describa una situación donde perdió trabajo, sobrescribió un archivo importante o no pudo identificar qué cambió entre una versión y otra de un documento o proyecto.

Producto esperado

Un documento breve (media página aproximadamente) con sus respuestas, que será socializado voluntariamente en un espacio de discusión grupal de 10 minutos guiado por el instructor.

3. Actividades de Contextualización

En esta fase se presentan los conceptos teóricos necesarios para comprender qué es Git, cómo se diferencia de otras herramientas y cuál es su papel dentro del ciclo de vida del desarrollo de software.

3.1 ¿Qué es Git?

Git es un sistema de control de versiones distribuido, creado en 2005 por Linus Torvalds (el creador del kernel de Linux). A diferencia de los sistemas centralizados, en Git cada persona que clona un repositorio tiene una copia completa del historial del proyecto en su propio equipo, lo que permite trabajar sin conexión permanente a un servidor y ofrece mayor seguridad ante la pérdida de información.

3.2 Conceptos clave antes de comenzar

- Repositorio (repo): carpeta de un proyecto que Git supervisa y de la cual guarda el historial de cambios.
- Commit: una "fotografía" o registro guardado del estado del proyecto en un momento determinado.
- Área de preparación (staging area): espacio intermedio donde se colocan los cambios que se desean incluir en el próximo commit.
- Working directory: los archivos tal como están actualmente en su carpeta de trabajo.
- Rama (branch): una línea de desarrollo independiente que permite trabajar en una funcionalidad sin afectar el código principal.

- Repositorio remoto: una copia del repositorio alojada en un servidor como GitHub, GitLab o Bitbucket.

3.3 Los tres estados de un archivo en Git

Todo archivo dentro de un repositorio Git puede encontrarse en tres estados posibles, y comprender este flujo es esencial para entender por qué se necesitan comandos como `git add` y `git status`:

1. Modificado (modified): el archivo cambió respecto a la última versión guardada, pero aún no ha sido marcado para el próximo commit.
2. Preparado (staged): el archivo fue marcado con `git add` y está listo para formar parte del próximo commit.
3. Confirmado (committed): el cambio ya quedó guardado de forma permanente en el historial del repositorio local.

3.4 Git y el flujo de trabajo colaborativo

En un equipo de desarrollo real, varias personas trabajan simultáneamente sobre el mismo proyecto. Sin un sistema como Git, coordinar estos cambios sería prácticamente imposible. Las ramas permiten que cada integrante del equipo desarrolle una funcionalidad de forma aislada y que, una vez terminada y probada, se integre (fusiona) al código principal sin poner en riesgo la estabilidad del proyecto.

Para tener en cuenta

La rama principal de un repositorio suele llamarse main (anteriormente master). Cualquier funcionalidad nueva debería desarrollarse en una rama distinta y fusionarse a main solo cuando esté lista y validada.

4. Actividades de Apropiación del Conocimiento

En esta fase usted aplicará, paso a paso y en su propio equipo, cada uno de los comandos trabajados en la guía. Siga las instrucciones en el orden indicado y registre evidencia (capturas de pantalla) de cada paso en su carpeta de evidencias.

4.1 git clone — Obtener una copia de un repositorio

git clone descarga una copia completa de un repositorio remoto (incluyendo todo su historial de commits) a su computador. Es normalmente el primer comando que se ejecuta cuando se va a trabajar sobre un proyecto existente.

Procedimiento paso a paso

1. Ingrese a GitHub y ubique el repositorio que desea clonar (por ejemplo, uno proporcionado por el instructor).
2. Haga clic en el botón verde "Code" y copie la URL en formato HTTPS.
3. Abra una terminal (Git Bash o la terminal integrada de VS Code) en la carpeta donde desea guardar el proyecto.
4. Ejecute el comando indicando la URL copiada:

```
$ git clone https://github.com/usuario/nombre-repositorio.git
```

5. Ingrese a la carpeta que se creó automáticamente con el nombre del repositorio:

```
$ cd nombre-repositorio
```

Verifique que el clonado fue exitoso listando el contenido de la carpeta (comando ls en Git Bash o dir en Windows) y confirmando que aparecen los archivos del proyecto original.

Explicación del comando

Al clonar, Git no solo copia los archivos actuales: también trae todo el historial de commits y crea automáticamente una conexión llamada 'origin' hacia el repositorio remoto, lo que le permitirá más adelante subir (push) y bajar (pull) cambios.

4.2 git status — Conocer el estado del repositorio

git status muestra en qué estado se encuentra su repositorio: qué archivos han sido modificados, cuáles están preparados (staged) para el próximo commit y cuáles todavía no están siendo rastreados por Git. Es el comando que más se utiliza en el día a día, porque evita cometer errores.

Procedimiento paso a paso

1. Dentro de la carpeta del repositorio clonado, cree o modifique un archivo cualquiera, por ejemplo un archivo llamado prueba.txt.
2. Ejecute el comando:

```
$ git status
```

3. Observe la salida. Si el archivo es nuevo, aparecerá en rojo bajo la sección "Untracked files" (archivos no rastreados).

4. Repita el comando después de cada paso importante (después de crear un archivo, después de usar git add, después de hacer un commit) para acostumbrarse a verificar constantemente el estado del repositorio.

Explicación del comando

git status no modifica nada: es un comando de solo consulta. Por eso se recomienda ejecutarlo con frecuencia — antes de agregar cambios, antes de confirmar un commit y antes de cambiar de rama — para evitar sorpresas.

4.3 git add . — Preparar los cambios para el commit

git add . mueve todos los archivos nuevos o modificados dentro de la carpeta actual (y

sus subcarpetas) hacia el área de preparación (staging area), dejándolos listos para ser incluidos en el próximo commit. El punto (.) indica "todos los cambios a partir de esta ubicación".

Procedimiento paso a paso

1. Después de crear o modificar uno o varios archivos, ejecute git status para confirmar qué cambios existen.
2. Ejecute el comando para preparar todos los cambios de una sola vez:

```
$ git add .
```

3. Vuelva a ejecutar git status. Ahora los archivos deberían aparecer en verde bajo la sección "Changes to be committed" (cambios listos para confirmar).
4. Si desea preparar un solo archivo en lugar de todos, puede usar git add nombre-del-archivo, pero para esta guía trabajaremos con git add . por practicidad.

Explicación del comando

El área de preparación le permite decidir exactamente qué cambios quedarán registrados en el próximo commit, incluso si tiene varios archivos modificados y solo quiere confirmar algunos de ellos. git add . es una forma rápida de preparar todo de una sola vez cuando todos los cambios pertenecen a la misma tarea.

4.4 git log — Consultar el historial de commits

Antes de ver git log, complete el ciclo confirmando el cambio preparado con un commit (este comando se incluye aquí como paso intermedio necesario para poder observar resultados en el historial):

```
$ git commit -m "Agrega archivo de prueba"
```

git log muestra el historial completo de commits del repositorio: quién hizo cada cambio, cuándo lo hizo, y qué mensaje describe ese cambio.

Procedimiento paso a paso

1. Ejecute el comando básico:

```
$ git log
```

2. Observe que cada commit muestra un identificador único (hash), el autor, la fecha y el mensaje del commit.
3. Para salir de la vista del historial (si su terminal queda en modo de paginación), presione la tecla q.
4. Pruebe también una versión resumida, muy usada en la práctica profesional:

```
$ git log --oneline
```

Explicación del comando

git log es la memoria del proyecto. Permite entender la evolución del código, identificar en qué momento se introdujo un error y saber quién realizó cada cambio, lo cual es esencial para la trazabilidad en proyectos de software.

4.5 Creación y uso de ramas (branches)

Una vez dominados los comandos básicos, es momento de introducir el trabajo con ramas, la herramienta que hace posible que varias personas —o una sola persona trabajando en varias funcionalidades— avancen en paralelo sin afectar el código estable del proyecto.

Paso 1: Consultar en qué rama se encuentra

Todo repositorio Git tiene al menos una rama activa. Para saber en cuál se encuentra actualmente:

```
$ git branch
```

La rama activa aparece marcada con un asterisco (*), generalmente llamada main.

Paso 2: Crear una nueva rama

Para crear una rama nueva sin cambiarse todavía a ella:

```
$ git branch nombre-de-la-rama
```

Por ejemplo, si va a desarrollar un formulario de inicio de sesión:

```
$ git branch feature-login
```

Paso 3: Cambiarse a la nueva rama

Para moverse a la rama recién creada y comenzar a trabajar en ella:

```
$ git checkout feature-login
```

También puede crear la rama y cambiarse a ella en un solo paso, combinando ambas acciones:

```
$ git checkout -b feature-login
```

Paso 4: Trabajar y confirmar cambios dentro de la rama

Dentro de la nueva rama, realice modificaciones en los archivos del proyecto y repita el flujo ya conocido:

```
$ git status
```

```
$ git add .
```

```
$ git commit -m "Agrega estructura inicial del formulario de login"
```

Paso 5: Verificar el historial de la rama

Use `git log` para confirmar que el commit quedó registrado dentro de esta rama:

```
$ git log --oneline
```

Paso 6: Regresar a la rama principal y fusionar los cambios

Cuando la funcionalidad está lista, se regresa a la rama principal:

```
$ git checkout main
```

Y se fusionan (merge) los cambios de la rama de trabajo hacia la rama principal:

```
$ git merge feature-login
```

Explicación del proceso

Trabajar con ramas evita que el código en desarrollo, que aún puede tener errores, afecte la versión estable del proyecto (main). Solo cuando una funcionalidad está terminada y probada se integra al código principal mediante un merge, lo que reduce el riesgo de romper el trabajo de otros integrantes del equipo.

5. Actividad de Transferencia (Sustentación)

Esta es la actividad final e integradora de la guía. Usted deberá demostrar, de manera individual, el dominio de los conceptos y comandos trabajados, transfiriendo lo aprendido a un escenario práctico nuevo, evaluado directamente por el instructor.

5.1 Descripción de la actividad

Usted creará un repositorio propio en GitHub (o utilizará uno asignado por el instructor) y realizará, en vivo y ante el instructor, el siguiente procedimiento completo, explicando en voz alta qué hace cada comando y por qué es necesario en ese punto del flujo de trabajo:

1. Clonar el repositorio en su equipo utilizando `git clone`.
2. Verificar el estado inicial del repositorio con `git status`.
3. Crear un archivo nuevo relacionado con una funcionalidad ficticia (por ejemplo, un archivo README de una nueva característica).
4. Preparar los cambios con `git add`.
5. Confirmar los cambios con un mensaje de commit claro y descriptivo.
6. Consultar el historial de commits con `git log`.
7. Crear una nueva rama y cambiarse a ella.
8. Realizar al menos un cambio y un commit dentro de esa rama.
9. Regresar a la rama principal y fusionar (merge) los cambios de la nueva rama.
10. Argumentar, con sus propias palabras, el aporte de cada comando al control de versiones y al trabajo colaborativo en un equipo de desarrollo de software.

5.2 Condiciones de la sustentación

- Tiempo estimado por aprendiz: entre 10 y 15 minutos.
- La sustentación puede realizarse de forma presencial ante el instructor o mediante una grabación de pantalla con audio explicativo.

- No se aceptan procedimientos copiados o explicados de memoria sin comprensión: el instructor podrá formular preguntas adicionales sobre cualquier comando ejecutado.
- Debe presentarse con el repositorio de práctica ya clonado y funcional antes de iniciar la sustentación.

5.3 Rúbrica de evaluación

La sustentación será evaluada por el instructor conforme a los siguientes criterios:

Criterio	Descripción a evaluar en la sustentación	Puntaje máx.	Puntaje obtenido
Dominio conceptual	El aprendiz explica con sus propias palabras qué es un repositorio, el área de preparación (staging) y el historial de commits.	20	
Ejecución de comandos	Demuestra en vivo el uso correcto de git clone, git status, git add . y git log sobre un repositorio real.	30	
Manejo de ramas	Crea, cambia y fusiona una rama nueva explicando el propósito de trabajar con ramas en un equipo de desarrollo.	25	
Argumentación del aporte	Sustenta con argumentos propios el valor de Git para la trazabilidad, el trabajo colaborativo y la calidad del software.	15	
Comunicación y evidencias	Presenta capturas de pantalla o video de la ejecución y responde preguntas del instructor con claridad.	10	
TOTAL		100	

5.4 Preguntas orientadoras para la argumentación final

Como cierre de la sustentación, prepárese para responder oralmente al menos dos de las siguientes preguntas, que integran la reflexión inicial con la práctica realizada:

- ¿Qué hubiera pasado si, en el escenario que sustentó, hubiera olvidado ejecutar git add . antes de hacer el commit?
- ¿Por qué cree que en un equipo de trabajo real no se debería hacer commit directamente sobre la rama main?
- ¿De qué manera git log le permitiría a un equipo identificar quién introdujo un error específico en el código?
- ¿Qué relación encuentra entre el uso de ramas y la calidad del software que se entrega a un cliente?

Nota final del instructor

El objetivo de esta guía no es memorizar comandos, sino comprender el propósito de cada uno dentro del ciclo de vida del desarrollo de software. Un desarrollador que entiende el "por qué" de Git resolverá con mayor facilidad los problemas de versionado que surgirán en su vida profesional.